

National University of Computer and Emerging Sciences, Islamabad
FAST School of Computing



Submitted by:

Hiba Asif (25I-0601)

Laiba Mubashir (25I-0722)

Juhayra Khan (25I-0922)

Bachelors of Computer Science

Course: MT1008-MULTIVARIABLE CALCULUS

Date of Submission: 05/05/26

Lecturer: Ms. Sumaira Azhar

Introduction

Modern 3D game engines are essentially implemented with multivariable calculus. Every particle movement, physics interaction, and graphic effect you see is a consequence of math operations.

This project implements the mathematical backend for a high level simplified 3D game engine. Specifically, the mathematical backend for terrain shading, rigid-body physics, particle energy transfer, and volumetric light. All of these tools are fed from a master scalar field defined as the Universal Game Asset Field, which provides the energy and mass density of the game:

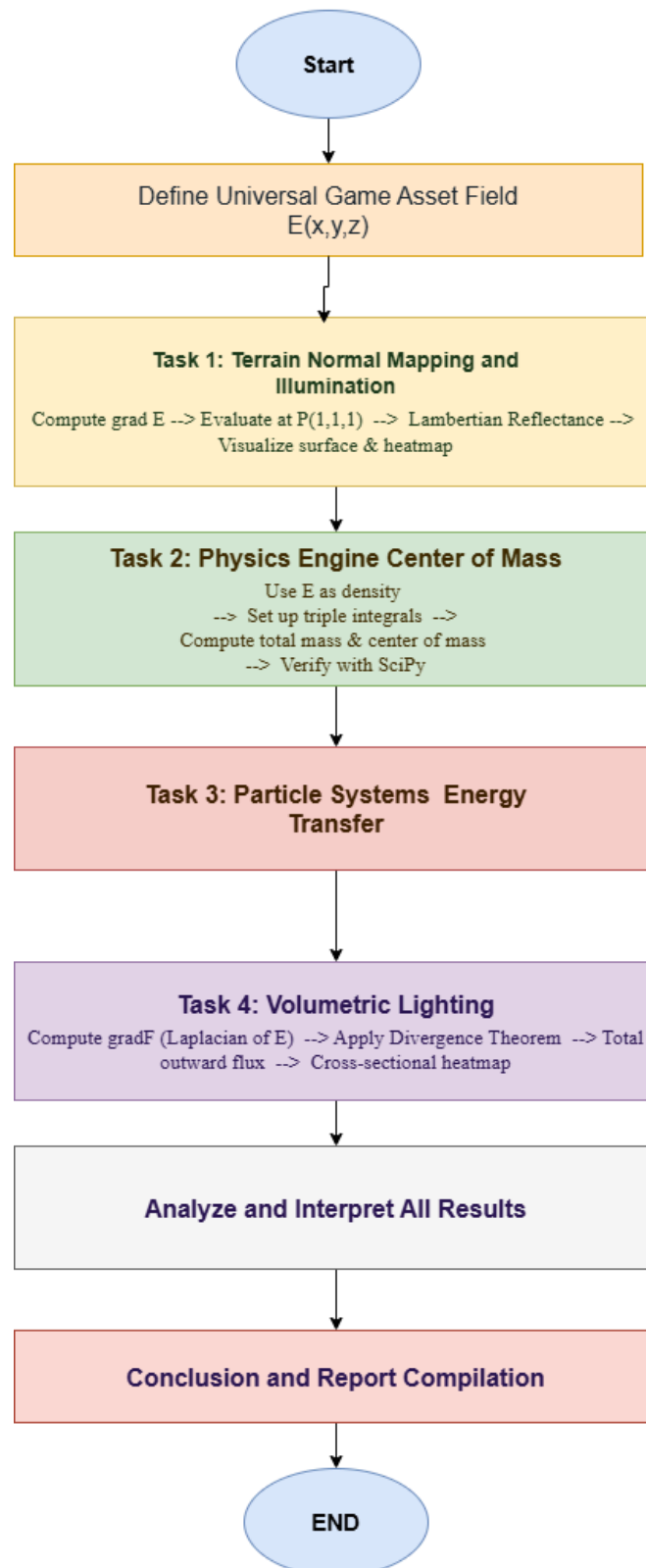
$$E(x,y,z) = x^2 y + 2y^2 z + 3z^2 x$$

Each task in this project takes in its inputs from $E(x,y,z)$.

The goal is to solve each task by hand, and then verify each one computationally in python. Symbolic manipulation is done in SymPy, numerical operations in NumPy, numerical integration with Scipy, and plots with Matplotlib.

The pipeline is simple. The gradient of the field does terrain shading. The field is used as a mass density map for the computation of center of mass. The gradient of the field is used as a force field for computing particle energy transfer, and the divergence of that force field is used as the volumetric lighting source.

Project Methodology Flowchart

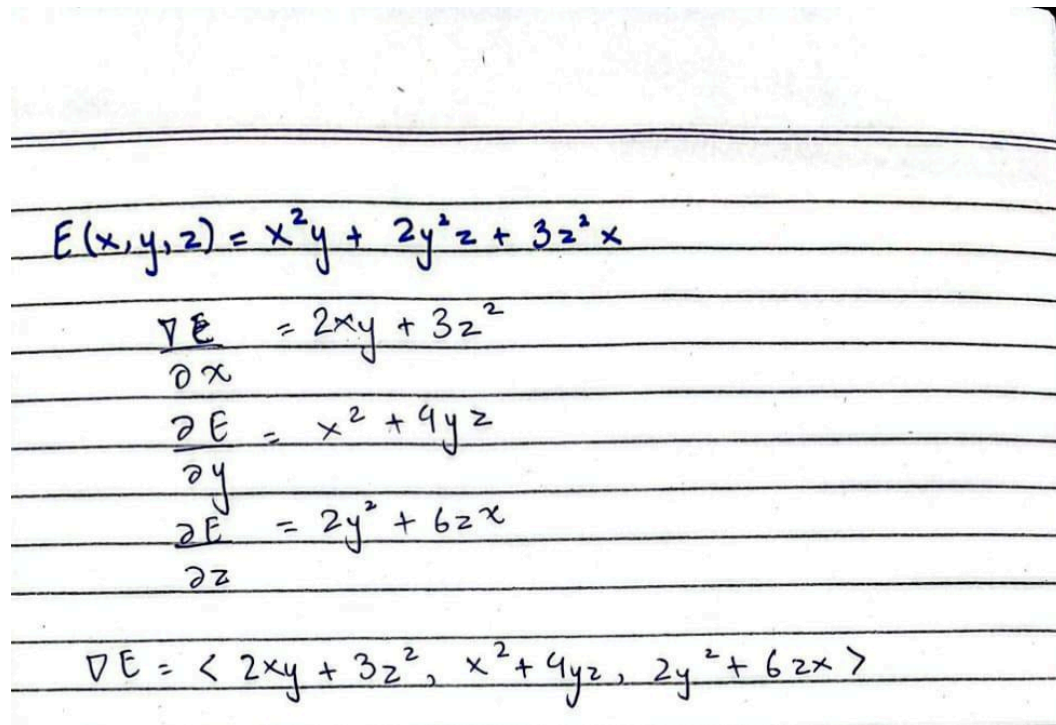


Task 1: Terrain Normal Mapping and Illumination

1.1 Background

In a 3D game, a realistic terrain must respond to light in a physically meaningful fashion. A surface pointed directly at a light source must be rendered bright, a surface tilted away from the light must be rendered darker. The gradient of the scalar field defining the terrain accomplishes this. At any point on a level surface, the gradient vector points perpendicularly outward, and is the surface normal. The dot product of that normal with a vector in the direction of incoming light determines then how much light that surface must be rendered with. This is known as Lambertian reflectance.

1.2 Analytical Computation of the Gradient



The image shows a handwritten derivation of the gradient of a scalar field $E(x, y, z) = x^2y + 2y^2z + 3z^2x$. The derivation is written on lined paper and includes the following steps:

$$E(x, y, z) = x^2y + 2y^2z + 3z^2x$$
$$\frac{\partial E}{\partial x} = 2xy + 3z^2$$
$$\frac{\partial E}{\partial y} = x^2 + 4yz$$
$$\frac{\partial E}{\partial z} = 2y^2 + 6zx$$
$$\nabla E = \langle 2xy + 3z^2, x^2 + 4yz, 2y^2 + 6zx \rangle$$

1.3 Evaluation at Point P(1, 1, 1)

$$\nabla E = \langle 2xy + 3z^2, x^2 + 4yz, 2y^2 + 6zx \rangle$$

at P(1, 1, 1):

$$\frac{\partial E}{\partial x} = 2(1)(1) + 3(1)^2 = 5$$

$$\frac{\partial E}{\partial y} = (1)^2 + 4(1)(1) = 5$$

$$\frac{\partial E}{\partial z} = 2(1)^2 + 6(1)(1) = 8$$

$$\nabla E \text{ at } P = \langle 5, 5, 8 \rangle$$

So this is surface normal vector

1.4 Lambertian Reflectance Calculation

Lambertian Reflectance:

Light vector $L = \langle 1, 1, 1 \rangle$

$$\text{Normalize } \nabla E: \text{magnitude} = \sqrt{25 + 25 + 64} = \sqrt{114}$$

$$\text{Normalize } L: \text{magnitude} = \sqrt{3}$$

$$\text{Dot product: } (5 \cdot 1 + 5 \cdot 1 + 8 \cdot 1) / (\sqrt{114} \cdot \sqrt{3}) = 0.973$$

This is reflectance value.

1.5 Python Verification and Visualization

```

import numpy as np
import matplotlib.pyplot as plt
from sympy import symbols, diff, sqrt, Matrix

# --- Symbolic gradient verification ---
x, y, z = symbols('x y z')
E = x**2 * y + 2*y**2 * z + 3*z**2 * x

dEdx = diff(E, x)
dEdy = diff(E, y)
dEdz = diff(E, z)

print("Gradient components:")
print("dE/dx =", dEdx)
print("dE/dy =", dEdy)
print("dE/dz =", dEdz)

# Evaluate at P(1,1,1)
grad_P = Matrix([dEdx.subs([(x,1),(y,1),(z,1)]),
                  dEdy.subs([(x,1),(y,1),(z,1)]),
                  dEdz.subs([(x,1),(y,1),(z,1)])])
print("\nGradient at P(1,1,1):", grad_P.T)

import numpy as np
import matplotlib.pyplot as plt
from sympy import symbols, diff, sqrt, Matrix
x, y, z = symbols('x y z')
E = x**2 * y + 2*y**2 * z + 3*z**2 * x

dEdx = diff(E, x)
dEdy = diff(E, y)
dEdz = diff(E, z)
print("Gradient components:")
print("dE/dx =", dEdx)
print("dE/dy =", dEdy)
print("dE/dz =", dEdz)
grad_P = Matrix([dEdx.subs([(x,1),(y,1),(z,1)]),
                  dEdy.subs([(x,1),(y,1),(z,1)]),
                  dEdz.subs([(x,1),(y,1),(z,1)])])
print("\nGradient at P(1,1,1):", grad_P.T)
# Lambertian reflectance
L = Matrix([1, 1, 1])
cos_theta = grad_P.dot(L) / (grad_P.norm() * L.norm())
print("Lambertian reflectance:", float(cos_theta))

```

```

# Plot 1: Surface near P with normal vector
fig = plt.figure(figsize=(14, 6))
ax1 = fig.add_subplot(121, projection='3d')
# Create grid around P(1,1,1) on the level surface E=6
# We fix z and vary x,y to approximate the surface
x_vals = np.linspace(0.5, 1.5, 30)
y_vals = np.linspace(0.5, 1.5, 30)
X, Y = np.meshgrid(x_vals, y_vals)
# Solve for Z approximately near P using E(x,y,z)=6
# At P(1,1,1), dE/dz = 8, so  $Z \approx 1 + (6 - E(x,y,1))/8$ 
Z_approx = 1 + (6 - (X**2 * Y + 2*Y**2 * 1 + 3*1**2 * X)) / 8
ax1.plot_surface(X, Y, Z_approx, alpha=0.5, cmap='viridis')
# Normal vector at P
ax1.quiver(1, 1, 1, 5, 5, 8, length=0.1, color='red', linewidth=2, label='Normal at P')
ax1.scatter(1, 1, 1, color='red', s=50, zorder=5)
ax1.set_title('Level Surface E=6 near P(1,1,1)\nwith Surface Normal')
ax1.set_xlabel('X'); ax1.set_ylabel('Y'); ax1.set_zlabel('Z')
ax1.legend()
# Plot 2: Heatmap of gradient magnitudes
ax2 = fig.add_subplot(122)
x_vals2 = np.linspace(0, 2, 100)
y_vals2 = np.linspace(0, 2, 100)

y_valsz = np.linspace(0, 2, 100)
X2, Y2 = np.meshgrid(x_vals2, y_vals2)
Z_fixed = 1 # fix z=1 for 2D slice
dEdx_num = 2*X2*Y2 + 3*Z_fixed**2
dEdy_num = X2**2 + 4*Y2*Z_fixed
dEdz_num = 2*Y2**2 + 6*Z_fixed*X2
grad_mag = np.sqrt(dEdx_num**2 + dEdy_num**2 + dEdz_num**2)
hmap = ax2.contourf(X2, Y2, grad_mag, levels=50, cmap='hot')
plt.colorbar(hmap, ax=ax2, label='Gradient Magnitude |∇E|')
ax2.set_title('Heatmap of Gradient Magnitudes\n(z=1 slice) - Brightness Map')
ax2.set_xlabel('X'); ax2.set_ylabel('Y')
ax2.plot(1, 1, 'bo', markersize=8, label='P(1,1,1)')
ax2.legend()
plt.tight_layout()
plt.savefig('task1_terrain.png', dpi=150)
plt.show()

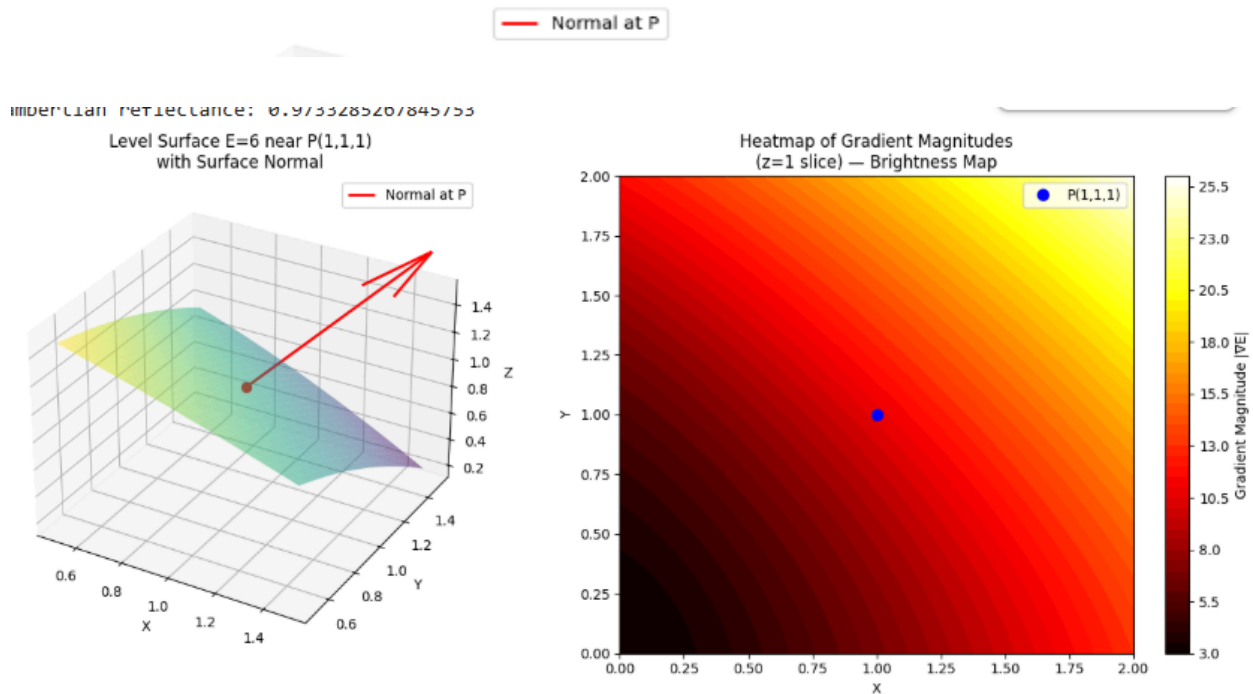
```

Output:

Gradient components:
 $dE/dx = 2*x*y + 3*z**2$
 $dE/dy = x**2 + 4*y*z$
 $dE/dz = 6*x*z + 2*y**2$

Gradient at P(1,1,1): Matrix([[5, 5, 8]])
 Lambertian reflectance: 0.9733285267845753

Level Surface E=6 near P(1,1,1)
 with Surface Normal



1.6 Interpretation

The gradient vector E at any point on the terrain surface tells the renderer which way the surface is facing. If it didn't know this direction, the engine would have no way to know how much light a surface was receiving, and the terrain would look completely flat with evenly lit surfaces that have no depth.

At point $P(1, 1, 1)$, the normal vector $5i + 5j + 8k$ indicates the surface is tilted outward and up on all 3 axes. The reflectance value of 0.973 indicates this surface is facing almost directly toward the light source, so the renderer modeled this as being nearly maximally lit. If the surface at P were perpendicular to the light direction, the reflectance would have been 0 and the point would have been completely black.

The heatmap extends this over the whole xy -plane at $z = 1$. Lighter areas are ones with a large gradient magnitude, indicating the field is changing quickly and the terrain is facing strongly toward the light. Darker areas are where the terrain is flatter or turned away from the light source. This evaluation of the normal per-point is precisely how modern game engines compute the diffuse lighting for their terrain, so the gradient is the core mathematical basis of the entire lighting system.

2. Physics Engine – Center of Mass (Triple Integrals)

2.1 Setting Up the Triple Integral for Total Mass

The mass density of the 3D game asset is given by the universal field itself, so $\delta(x, y, z) = E(x, y, z) = x^2y + 2y^2z + 3z^2x$. The total mass M is computed by integrating this density over the bounding volume $[0, 2] \times [0, 2] \times [0, 2]$:

Bounding volume = $(0, 0, 0) \rightarrow (2, 2, 2)$

$\therefore$ limits for all = $0 \rightarrow 2$

$$\therefore M = \int_0^2 \int_0^2 \int_0^2 (x^2y + 2y^2z + 3z^2x) dx dy dz$$

$$M = \int_0^2 \int_0^2 \left[\frac{x^3y}{3} + \frac{2y^2xz^2}{2} + \frac{3z^2x^2}{2} \right]_0^2 dy dz$$

$$M = \int_0^2 \int_0^2 \left[\frac{8}{3}y + 4y^2z + 6z^2 \right] dy dz$$

$$M = \int_0^2 \left[\frac{4y^2}{3} + \frac{4}{3}y^3z + 6yz^2 \right]_0^2 dz$$

$$M = \int_0^2 \left[\frac{16}{3} + \frac{32z}{3} + 12z^2 \right] dz$$

$$M = \left[\frac{16z}{3} + \frac{16z^2}{3} + 4z^3 \right]_0^2$$

$$M = \frac{32}{3} + \frac{64}{3} + 32$$

$$= \frac{96}{3} + 32 = 32 + 32, \therefore \boxed{M_{\text{tot}} = 64}$$

Total Mass $M = 64$

2.2 Center of Mass Integrals

The coordinates of the center of mass are found by computing the first moment integrals and dividing each by the total mass $M = 64$. The three moment integrals are:

Center of Mass = (x_c, y_c, z_c) where:

$$x_c = \int_0^2 \int_0^2 \int_0^2 x \cdot E \, dx \, dy \, dz$$

$$y_c = \int_0^2 \int_0^2 \int_0^2 y \cdot E \, dx \, dy \, dz$$

$$z_c = \int_0^2 \int_0^2 \int_0^2 z \cdot E \, dx \, dy \, dz$$

$$x_c = \int_0^2 \int_0^2 \int_0^2 x^3 y + 2xy^2 z + 3z^2 x^2 \, dx \, dy \, dz$$

$$x_c = \int_0^2 \int_0^2 \left[\frac{x^4 y}{4} + 2x^2 y^2 z + z^2 x^3 \right]_0^2 dy \, dz$$

$$x_c = \int_0^2 \int_0^2 4y + 4y^2 z + 8z^2 \, dy \, dz$$

$$x_c = \int_0^2 \left[2y^2 + \frac{4}{3} y^3 z + 8z^2 y \right]_0^2 dz$$

$$x_c = \int_0^2 8 + \frac{32z}{3} + 16z^2 \, dz$$

$$x_c = \int_0^2 \left[8z + \frac{16z^2}{3} + \frac{16z^3}{3} \right]_0^2 dz$$

$$x_c = 16 + \frac{64}{3} + \frac{128}{3} = 16 + \frac{192}{3} = \boxed{80}$$

$$y_c = \int_0^2 \int_0^2 \int_0^2 x^2 y^2 + 2y^3 z + 3z^2 xy \, dx \, dy \, dz$$

$$y_c = \int_0^2 \int_0^2 \left[\frac{x^3 y^2}{3} + \frac{2x y^3 z}{2} + \frac{3z^2 x^2 y}{2} \right]_0^2 dy \, dz$$

$$y_c = \int_0^2 \int_0^2 \frac{8}{3} y^2 + 4y^3 z + 6z^2 y \, dy \, dz$$

$$y_c = \int_0^2 \left[\frac{8}{3} y^3 + y^4 z + 3z^2 y^2 \right]_0^2 dz$$

$$y_c = \int_0^2 \left\{ \frac{64}{3} + 16z + 12z^2 \right\} dz$$

$$y_c = \left[\frac{64z}{3} + 8z^2 + 4z^3 \right]_0^2$$

$$= \frac{128}{3} + 32 + 32 = \frac{128 + 96}{3} = \boxed{704/3}$$

$$z_c = \int_0^2 \int_0^2 \int_0^2 x^2 y z + 2y^2 z^2 + 3z^3 x \, dx \, dy \, dz$$

$$z_c = \int_0^2 \int_0^2 \left[\frac{x^3 y z}{3} + \frac{2}{3} y^2 z^3 + \frac{3}{4} z^4 x^2 \right]_0^2 dy \, dz$$

$$z_c = \int_0^2 \int_0^2 \frac{8}{3} y z + 4y^2 z^2 + 6z^3 y \, dy \, dz$$

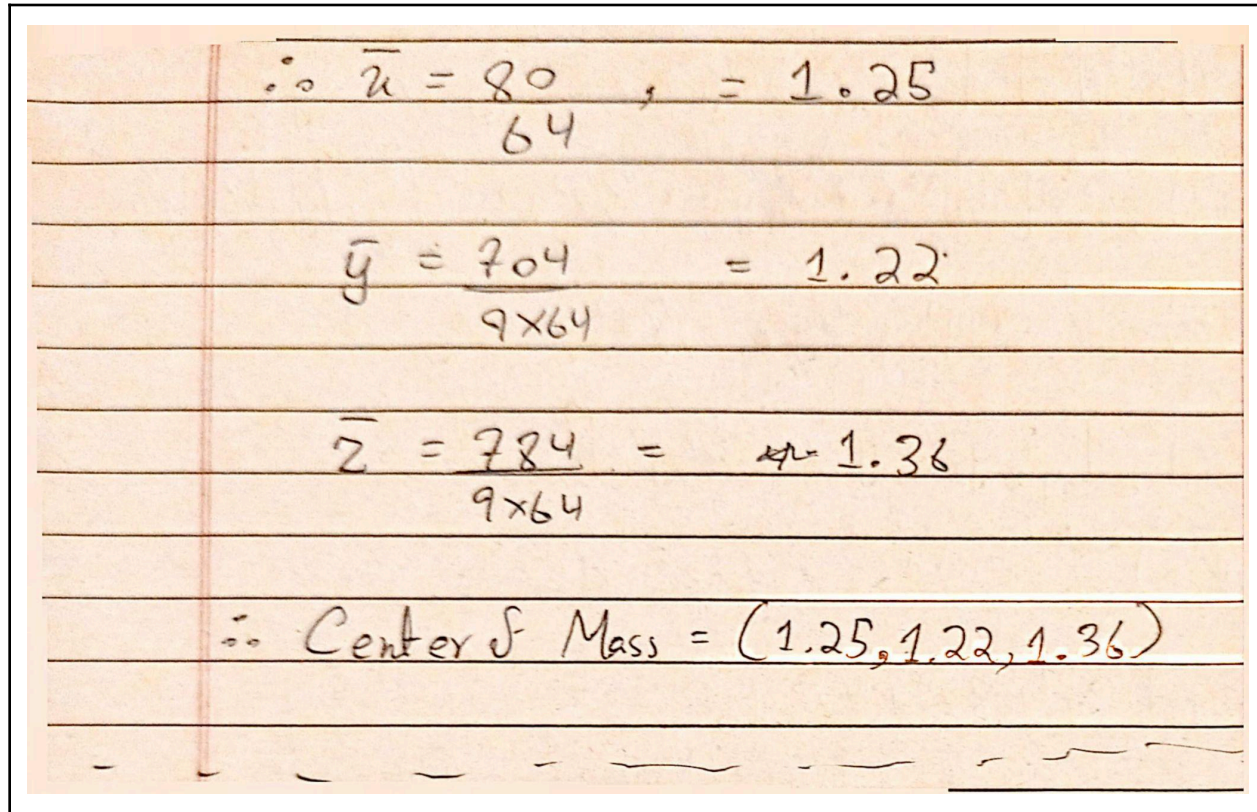
$$z_c = \int_0^2 \left[\frac{4}{3} y^2 z + \frac{4}{3} y^3 z^2 + 6z^3 y \right]_0^2 dz$$

$$z_c = \int_0^2 \left\{ \frac{16}{3} z + \frac{32}{3} z^2 + 12z^3 \right\} dz$$

$$z_c = \left[\frac{8}{3} z^2 + \frac{32}{9} z^3 + 3z^4 \right]_0^2$$

$$= \frac{32}{3} + \frac{256}{9} + 48 = \boxed{\frac{784}{3}}$$

The center of mass coordinates are therefore:



Handwritten calculations on lined paper:

$$\bar{x} = \frac{80}{64} = 1.25$$
$$\bar{y} = \frac{704}{9 \times 64} = 1.22$$
$$\bar{z} = \frac{784}{9 \times 64} = 1.36$$

$\therefore$ Center of Mass = (1.25, 1.22, 1.36)

Center of Mass $\approx$ (1.25, 1.222, 1.361)

2.3 Python Code and Numerical Verification

```
import sympy as sp
from scipy.integrate import tplquad
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D
```

```
x, y, z = sp.symbols('x y z')
```

```
E = x**2*y + 2*y**2*z + 3*z**2*x
```

```
total_mass = sp.integrate(
    sp.integrate(
        sp.integrate(E, (x, 0, 2)),
        (y, 0, 2)),
    (z, 0, 2))
```

```
print(f"Total mass (symbolic): {total_mass}")
```

```
Mx = sp.integrate(sp.integrate(sp.integrate(x*E, (x,0,2)), (y,0,2)), (z,0,2))
```

```
My = sp.integrate(sp.integrate(sp.integrate(y*E, (x,0,2)), (y,0,2)), (z,0,2))
```

```

Mz = sp.integrate(sp.integrate(sp.integrate(z*E, (x,0,2)), (y,0,2)), (z,0,2))

x_bar = Mx / total_mass
y_bar = My / total_mass
z_bar = Mz / total_mass
print(f"Center of mass (symbolic): ({x_bar}, {y_bar}, {z_bar})")

def density(zv, yv, xv):
    return xv**2*yv + 2*yv**2*zv + 3*zv**2*xv

def density_x(zv, yv, xv):
    return xv * (xv**2*yv + 2*yv**2*zv + 3*zv**2*xv)

def density_y(zv, yv, xv):
    return yv * (xv**2*yv + 2*yv**2*zv + 3*zv**2*xv)

def density_z(zv, yv, xv):
    return zv * (xv**2*yv + 2*yv**2*zv + 3*zv**2*xv)

M_num, _ = tplquad(density, 0, 2, 0, 2, 0, 2)
Mx_num, _ = tplquad(density_x, 0, 2, 0, 2, 0, 2)
My_num, _ = tplquad(density_y, 0, 2, 0, 2, 0, 2)
Mz_num, _ = tplquad(density_z, 0, 2, 0, 2, 0, 2)

print(f"Total mass (scipy): {M_num:.6f}")
print(f"Center of mass (scipy): ({Mx_num/M_num:.6f}, {My_num/M_num:.6f}, {Mz_num/M_num:.6f})")

fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')

corners = [
    [0,0,0],[2,0,0],[2,2,0],[0,2,0],
    [0,0,2],[2,0,2],[2,2,2],[0,2,2]
]

edges = [(0,1),(1,2),(2,3),(3,0),
          (4,5),(5,6),(6,7),(7,4),
          (0,4),(1,5),(2,6),(3,7)]

for e in edges:
    xs = [corners[e[0]][0], corners[e[1]][0]]
    ys = [corners[e[0]][1], corners[e[1]][1]]
    zs = [corners[e[0]][2], corners[e[1]][2]]
    ax.plot(xs, ys, zs, 'b-', alpha=0.4, linewidth=1.5)

cx, cy, cz = float(x_bar), float(y_bar), float(z_bar)
ax.scatter([cx], [cy], [cz], color='red', s=150, zorder=10,
           label=f'CoM ({cx:.2f}, {cy:.2f}, {cz:.2f})')
ax.text(cx+0.1, cy+0.1, cz+0.1,
        f'CoM\n({cx:.2f},{cy:.2f},{cz:.2f})', fontsize=9, color='red')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

```

```

ax.set_title('3D Bounding Volume  $[0,2]^3$  with Center of Mass')
ax.legend()
plt.tight_layout()
plt.show()

```

Python output:

```

Total mass (symbolic): 64
Center of mass (symbolic): (5/4, 11/9, 49/36)
Total mass (scipy): 64.000000
Center of mass (scipy): (1.250000, 1.222222, 1.361111)

```

2.4 Visualization

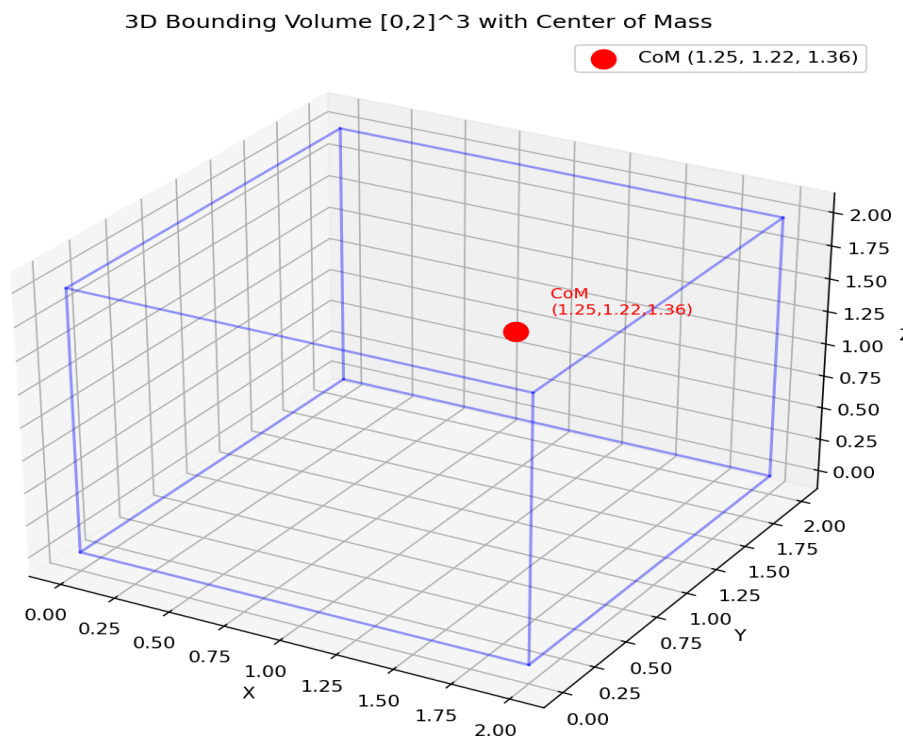


Figure 2: The cubic bounding volume $[0, 2]^3$ with the computed center of mass marked at $(1.25, 1.222, 1.361)$ shown as a red dot.

2.5 Interpretation

The center of mass is located at approximately $(1.25, 1.22, 1.36)$, which is slightly above and toward the higher-coordinate corner of the cube. This makes physical sense because the density function $E(x, y, z)$ is not symmetric in x , y , and z . The z -component has the highest contribution

from the $3z^2x$ term, which grows quickly, pulling the center of mass upward. The x-component is also slightly elevated for the same reason.

In a game physics engine, the center of mass is the point through which gravity acts and about which the object rotates when a torque is applied. If a force is applied at any point other than the center of mass, it will both translate and rotate the object. Knowing the exact location of the center of mass allows the engine to accurately compute torques, angular momentum, and rotational inertia. For a non-uniform object like this one, using an incorrect or estimated center of mass would cause visibly wrong spinning or tumbling behavior.

3. Particle Systems and Energy Transfer (Vector Fields and Line Integrals)

3.1 The Force Field $\mathbf{F} = \nabla E$

The gravity well of the game world is modeled by the gradient of the universal field. The resulting force field is:

$$\mathbf{F} = \nabla E = \langle 2xy + 3z^2, x^2 + 4yz, 6xz + 2y^2 \rangle$$

3.2 Checking if $\mathbf{F}$ is Conservative Using the Curl

A vector field is conservative if and only if its curl is the zero vector. The curl of $\mathbf{F}$ is as follows:

$$F = \nabla E = (2xy + 3z^2)\hat{i} + (x^2 + 4yz)\hat{j} + (6xz + 2y^2)\hat{k}$$

Curl :

$$\begin{array}{c|c|c} \hat{i} & \hat{j} & \hat{k} \\ \hline \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ \hline F_x & F_y & F_z \end{array}$$

$$\hat{k} = \left((x^2 + 4yz) \frac{\partial}{\partial x} - (2xy + 3z^2) \frac{\partial}{\partial y} \right) \hat{k}$$

$$= (2x - 2x) \hat{k} = 0 \hat{k}$$

$$\hat{j} = \left((2xy + 3z^2) \frac{\partial}{\partial z} - (6xz + 2y^2) \frac{\partial}{\partial x} \right) \hat{j}$$

$$= (6z - 6z) \hat{j} = 0 \hat{j}$$

$$\hat{i} = \left((x^2 + 4yz) \frac{\partial}{\partial z} - (6xz + 2y^2) \frac{\partial}{\partial y} \right) \hat{i}$$

$$= (4y - 4y) \hat{i} = 0 \hat{i}$$

$\Rightarrow F$ is a conservative vector field.

Because $F = \nabla E$, this result was expected. Any gradient field is automatically conservative. The curl being zero simply confirms that E is the scalar potential function for this force field.

3.3 Line Integral by the Fundamental Theorem

Since F is conservative with potential function $\phi(x, y, z) = E(x, y, z)$, the Fundamental Theorem of Line Integrals states that the line integral along any path C from point A to point B is:

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \phi(\mathbf{B}) - \phi(\mathbf{A}) = E(\mathbf{B}) - E(\mathbf{A})$$

The spaceship travels from $A = (0, 0, 0)$ to $B = (2, 2, 2)$. Evaluating the potential at both endpoints:

The image shows handwritten work on lined paper. It starts with the equation $\int_C \mathbf{F} \cdot d\mathbf{r} = E(b) - E(a)$. Then it defines $a = (0, 0, 0)$ and $b = (2, 2, 2)$. It then calculates $E(a) = 0$. Next, it calculates $E(b) = (2^2)(2) + (2)(2^2)(2) + 3(2^2)(2) = 48$. Finally, it concludes $\int_C \mathbf{F} \cdot d\mathbf{r} = 48 - 0$ and boxes the result 48 .

$$\int_C \mathbf{F} \cdot d\mathbf{r} = E(b) - E(a)$$
$$a = (0, 0, 0), \quad b = (2, 2, 2)$$
$$\therefore E(a) = 0$$
$$E(b) = (2^2)(2) + (2)(2^2)(2) + 3(2^2)(2)$$
$$= 48$$
$$\therefore \int_C \mathbf{F} \cdot d\mathbf{r} = 48 - 0$$
$$\boxed{48}$$

$$\int_C \mathbf{F} \cdot d\mathbf{r} = 48$$

3.4 Python Code

```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x, y, z = sp.symbols('x y z')

E = x**2*y + 2*y**2*z + 3*z**2*x

Fx = sp.diff(E, x)
Fy = sp.diff(E, y)
Fz = sp.diff(E, z)
print(f"F = ({Fx}, {Fy}, {Fz})")

curl_x = sp.diff(Fz, y) - sp.diff(Fy, z)
curl_y = sp.diff(Fx, z) - sp.diff(Fz, x)
curl_z = sp.diff(Fy, x) - sp.diff(Fx, y)

print(f"Curl of F: ({sp.simplify(curl_x)}, {sp.simplify(curl_y)}, {sp.simplify(curl_z)})")
print("Since curl F = (0, 0, 0), F is conservative.")

E_start = E.subs([(x,0), (y,0), (z,0)])
E_end = E.subs([(x,2), (y,2), (z,2)])
work = E_end - E_start
print(f"E(0,0,0) = {E_start}")
print(f"E(2,2,2) = {E_end}")
print(f"Total energy transferred = {work}")

Fx_num = sp.lambdify((x,y,z), Fx, 'numpy')
Fy_num = sp.lambdify((x,y,z), Fy, 'numpy')
Fz_num = sp.lambdify((x,y,z), Fz, 'numpy')

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

t = np.linspace(0, 2, 100)
ax.plot(t, t, t, 'r-', linewidth=3, label='Spaceship path')
ax.scatter([0], [0], [0], color='green', s=100, zorder=10, label='Start (0,0,0)')
ax.scatter([2], [2], [2], color='blue', s=100, zorder=10, label='End (2,2,2)')

gi = np.linspace(0, 2, 5)
GX, GY, GZ = np.meshgrid(gi, gi, gi)
U = Fx_num(GX, GY, GZ)
V = Fy_num(GX, GY, GZ)
W = Fz_num(GX, GY, GZ)

mag = np.sqrt(U**2 + V**2 + W**2) + 1e-10
ax.quiver(GX, GY, GZ,
          U/mag*0.3, V/mag*0.3, W/mag*0.3,
          color='gray', alpha=0.5, length=0.25, linewidth=0.8)

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title("Spaceship Path and Force Field  $F = \nabla E$ ")
ax.legend()
plt.tight_layout()
plt.show()
```

Python output:

```
F = (2*x*y + 3*z**2, x**2 + 4*y*z, 6*x*z + 2*y**2)
Curl of F: (0, 0, 0)
Since curl F = (0, 0, 0), F is conservative.
E(0,0,0) = 0
E(2,2,2) = 48
Total energy transferred = 48
```

3.5 Visualization

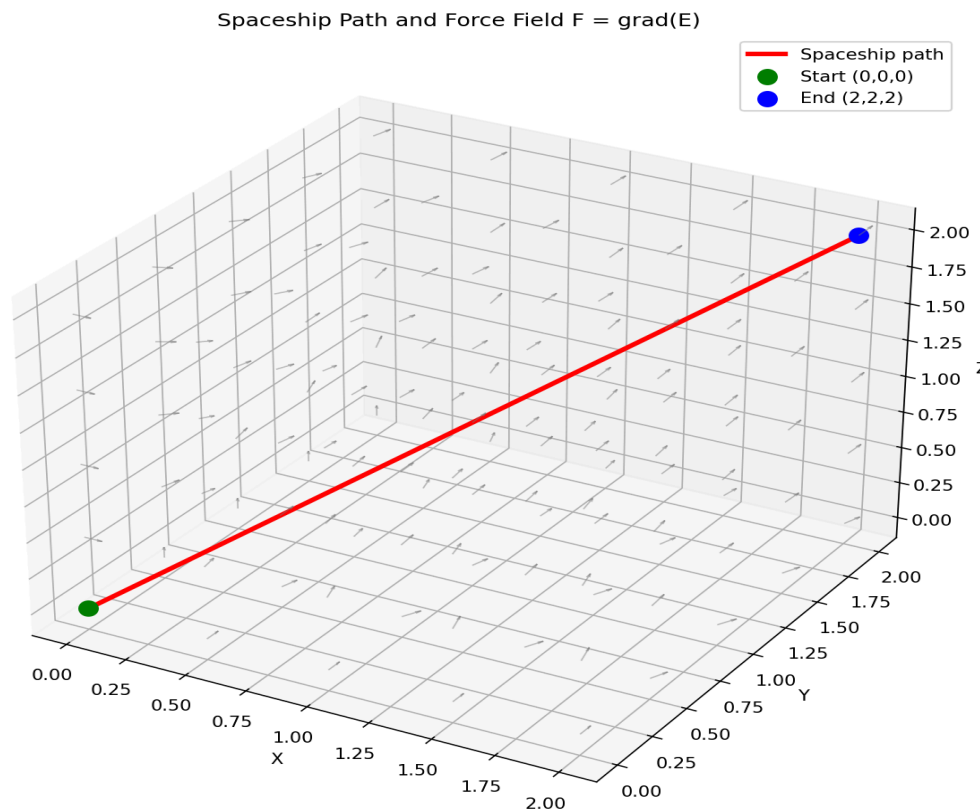


Figure 3: The spaceship's 3D flight path from $(0,0,0)$ to $(2,2,2)$ shown in red, overlaid on a quiver plot of the force field $F = \nabla E$ sampled on a $5 \times 5 \times 5$ grid.

3.6 Interpretation

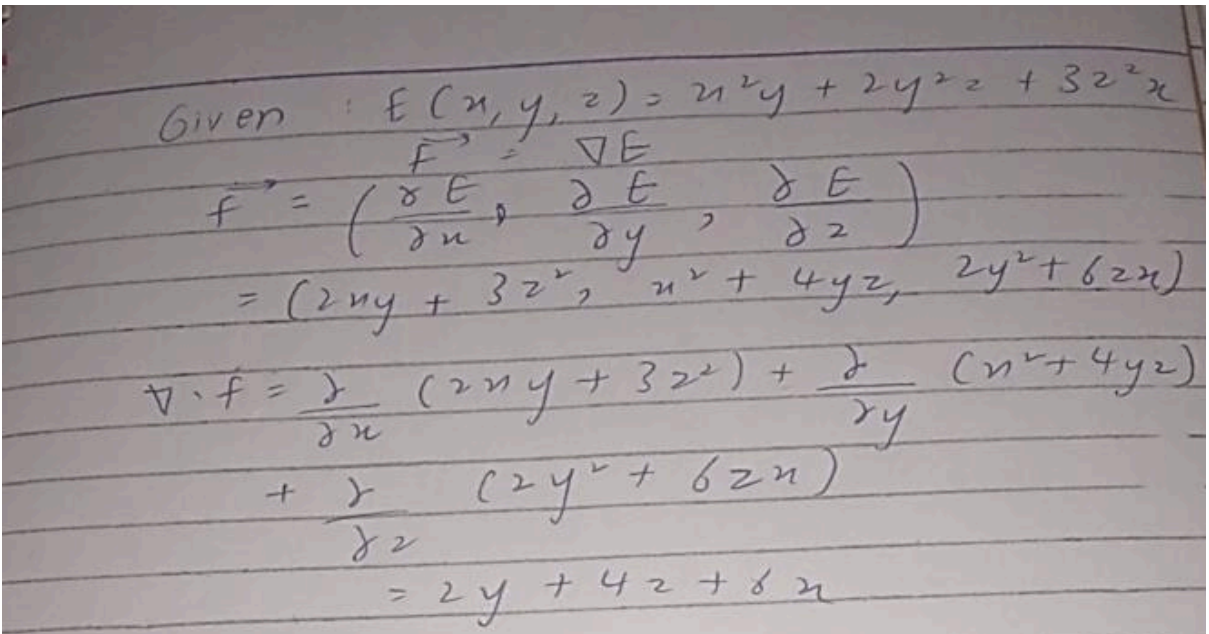
The total energy transferred to the spaceship as it travels from the origin to $(2,2,2)$ is 48 units. Because F is a conservative field, this value is completely independent of which path the spaceship takes. Whether it flies in a straight line, curves around obstacles, or spirals upward, the total energy change from start to end will always be 48.

From a gameplay perspective, this result has direct implications for balancing. If this gravity well field is used as a potential energy landscape, then a player starting at the origin and moving to (2,2,2) always gains exactly 48 units of kinetic energy from the field. This makes it easy to design fair challenge structures because the energy gained is path-independent. On the other hand, the direction of the force field shown in the quiver plot tells game designers where the force is strongest and how a particle or vehicle will be pulled as it moves through the world.

4. Volumetric Lighting (Divergence Theorem)

4.1 Computing the Divergence of F

The light energy field is identical to the gravity well field $F = \nabla E$. The divergence of F measures how much the field is spreading out or converging at each point. Since F is the gradient of a scalar field, its divergence is the laplacian of E:



Handwritten derivation of the divergence of F:

$$\begin{aligned} \text{Given } E(x, y, z) &= x^2y + 2y^2z + 3z^2x \\ \vec{F} &= \nabla E \\ \vec{F} &= \left(\frac{\partial E}{\partial x}, \frac{\partial E}{\partial y}, \frac{\partial E}{\partial z} \right) \\ &= (2xy + 3z^2, x^2 + 4yz, 2y^2 + 6zx) \\ \nabla \cdot \vec{F} &= \frac{\partial}{\partial x} (2xy + 3z^2) + \frac{\partial}{\partial y} (x^2 + 4yz) \\ &\quad + \frac{\partial}{\partial z} (2y^2 + 6zx) \\ &= 2y + 4z + 6x \end{aligned}$$

$$\text{div}(\vec{F}) = 2y + 4z + 6x$$

4.2 Applying the Divergence Theorem

The divergence theorem relates the total outward flux of a vector field through a closed surface S to the volume integral of its divergence inside the enclosed region V:

$$\begin{aligned}
 \iiint_S \vec{F} \cdot \hat{n} dS &= \iiint_V (\nabla \cdot \vec{F}) dV \\
 &= \int_0^2 \int_0^2 \int_0^2 (2y + 4z + 6x) dx dy dz \\
 &= \int_0^2 (2y + 4z + 6x) dx \\
 &= 4y + 8z + 12 \\
 &= \int_0^2 (4y + 8z + 12) dy \\
 &= 32 + 16z \\
 &= \int_0^2 (32 + 16z) dz \\
 &= 64 + 32 \\
 &= 96
 \end{aligned}$$

Total Outward flux = 96

Total Outward Flux = 96

4.3 Python Code and Numerical Verification

```

import sympy as sp
from scipy.integrate import tplquad

x, y, z = sp.symbols("x y z")
E = x**2*y + 2*y**2*z + 3*z**2*x

# gradient of E gives F
Fx = sp.diff(E, x)
Fy = sp.diff(E, y)
Fz = sp.diff(E, z)

# divergence of F is the Laplacian of E
div_F = sp.diff(Fx, x) + sp.diff(Fy, y) + sp.diff(Fz, z)
# div_F = 6x + 2y + 4z

# volume integral of divergence over [0,2]^3
flux = sp.integrate(sp.integrate(
    sp.integrate(div_F, (x,0,2)), (y,0,2)), (z,0,2))

```



```
# flux = 96

# numerical check with scipy
def div_func(zv, yv, xv):
    return 2*yv + 4*xv + 6*xv # the divergence expression

flux_num, err = tplquad(div_func, 0, 2, 0, 2, 0, 2)
# flux_num = 96.000000 (error estimate: 1.07e-12)
```

Python output:

$\text{div}(\mathbf{F}) = \text{Laplacian of } E = 6x + 2y + 4z$

Total flux (symbolic): 96

Total flux (scipy): 96.000000

Error estimate: 1.07e-12

4.4 Visualization

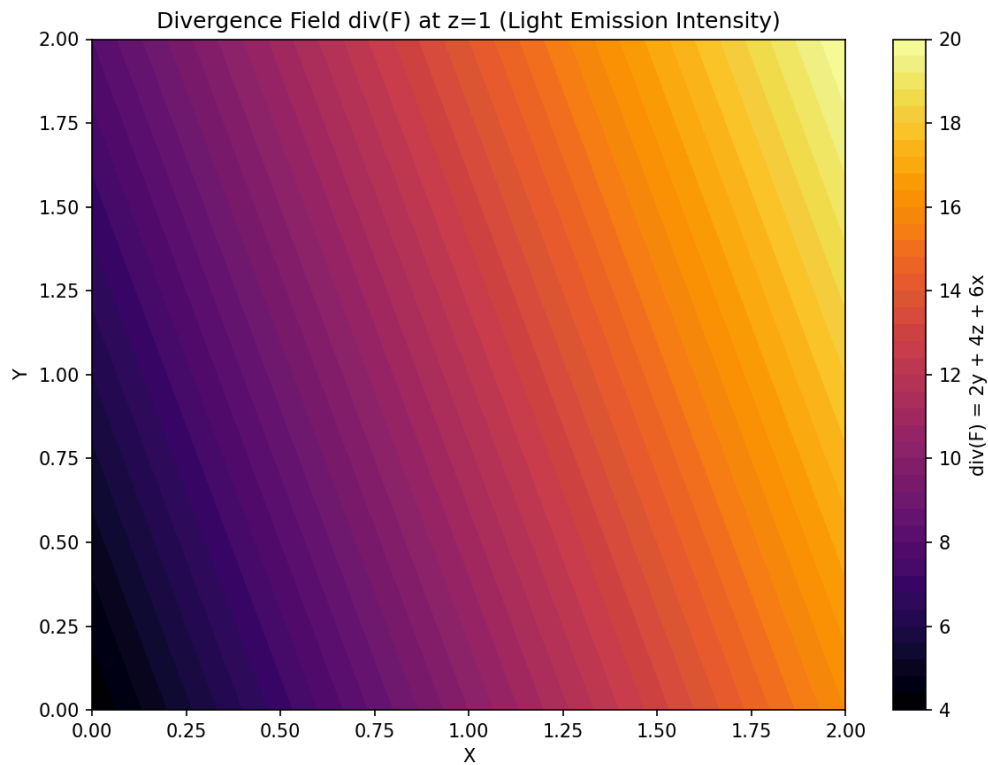


Figure 4: Heatmap of $\text{div}(\mathbf{F}) = 6x + 2y + 4z$ across the xy -plane at $z = 1$. Brighter regions represent higher light emission intensity.

4.5 Interpretation

The total outward flux of 96 units means that the magic artifact emits 96 units of net light energy outward through the six faces of the bounding cube every cycle. The divergence theorem is especially useful here because computing the surface integrals across all six faces of the cube separately would be tedious, while the volume integral of the divergence can be evaluated in a few direct steps.

The heatmap in Figure 4 shows the divergence at the $z = 1$ cross-section. The brightest region is in the upper right corner where x and y are both large, because the divergence $6x + 2y + 4z$ increases linearly in all three directions. This tells the game engine that the artifact emits more light in regions of high x and y , rather than emitting uniformly. In a volumetric lighting system, this divergence field directly controls how much light energy is added to each voxel of the scene, creating the appearance of glowing volumes with non-uniform intensity distributions.

This behavior is useful in volumetric rendering systems where space is divided into voxels. Each voxel receives the light energy proportional to the divergence value which allows the engine to simulate glowing regions, light scattering, and nonuniform illumination. This phenomena produces more realistic lighting effects as compared to the uniform emission models .

5. Python Annexure:

Part 1

```

import numpy as np
import matplotlib.pyplot as plt
from sympy import symbols, diff, sqrt, Matrix

# --- Symbolic gradient verification ---
x, y, z = symbols('x y z')
E = x**2 * y + 2*y**2 * z + 3*z**2 * x

dEdx = diff(E, x)
dEdy = diff(E, y)
dEdz = diff(E, z)

print("Gradient components:")
print("dE/dx =", dEdx)
print("dE/dy =", dEdy)
print("dE/dz =", dEdz)

# Evaluate at P(1,1,1)
grad_P = Matrix([dEdx.subs([(x,1),(y,1),(z,1)]),
                  dEdy.subs([(x,1),(y,1),(z,1)]),
                  dEdz.subs([(x,1),(y,1),(z,1)])])
print("\nGradient at P(1,1,1):", grad_P.T)

```

```

import numpy as np
import matplotlib.pyplot as plt
from sympy import symbols, diff, sqrt, Matrix
x, y, z = symbols('x y z')
E = x**2 * y + 2*y**2 * z + 3*z**2 * x

dEdx = diff(E, x)
dEdy = diff(E, y)
dEdz = diff(E, z)
print("Gradient components:")
print("dE/dx =", dEdx)
print("dE/dy =", dEdy)
print("dE/dz =", dEdz)
grad_P = Matrix([dEdx.subs([(x,1),(y,1),(z,1)]),
                  dEdy.subs([(x,1),(y,1),(z,1)]),
                  dEdz.subs([(x,1),(y,1),(z,1)])])
print("\nGradient at P(1,1,1):", grad_P.T)
# Lambertian reflectance
L = Matrix([1, 1, 1])
cos_theta = grad_P.dot(L) / (grad_P.norm() * L.norm())
print("Lambertian reflectance:", float(cos_theta))

# Plot 1: Surface near P with normal vector
fig = plt.figure(figsize=(14, 6))
ax1 = fig.add_subplot(121, projection='3d')
# Create grid around P(1,1,1) on the level surface E=6
# We fix z and vary x,y to approximate the surface
x_vals = np.linspace(0.5, 1.5, 30)
y_vals = np.linspace(0.5, 1.5, 30)
X, Y = np.meshgrid(x_vals, y_vals)
# Solve for Z approximately near P using E(x,y,z)=6
# At P(1,1,1), dE/dz = 8, so  $Z \approx 1 + (6 - E(x,y,1))/8$ 
Z_approx = 1 + (6 - (X**2 * Y + 2*Y**2 * 1 + 3*1**2 * X)) / 8
ax1.plot_surface(X, Y, Z_approx, alpha=0.5, cmap='viridis')
# Normal vector at P
ax1.quiver(1, 1, 1, 5, 5, 8, length=0.1, color='red', linewidth=2, label='Normal at P')
ax1.scatter(1, 1, 1, color='red', s=50, zorder=5)
ax1.set_title('Level Surface E=6 near P(1,1,1)\nwith Surface Normal')
ax1.set_xlabel('X'); ax1.set_ylabel('Y'); ax1.set_zlabel('Z')
ax1.legend()
# Plot 2: Heatmap of gradient magnitudes
ax2 = fig.add_subplot(122)
x_vals2 = np.linspace(0, 2, 100)
y_vals2 = np.linspace(0, 2, 100)

```



```

y_vals2 = np.linspace(0, 2, 100)
X2, Y2 = np.meshgrid(x_vals2, y_vals2)
Z_fixed = 1 # fix z=1 for 2D slice
dEdx_num = 2*X2*Y2 + 3*Z_fixed**2
dEdy_num = X2**2 + 4*Y2*Z_fixed
dEdz_num = 2*Y2**2 + 6*Z_fixed*X2
grad_mag = np.sqrt(dEdx_num**2 + dEdy_num**2 + dEdz_num**2)
hmap = ax2.contourf(X2, Y2, grad_mag, levels=50, cmap='hot')
plt.colorbar(hmap, ax=ax2, label='Gradient Magnitude |∇E|')
ax2.set_title('Heatmap of Gradient Magnitudes\n(z=1 slice) - Brightness Map')
ax2.set_xlabel('X'); ax2.set_ylabel('Y')
ax2.plot(1, 1, 'bo', markersize=8, label='P(1,1,1)')
ax2.legend()
plt.tight_layout()
plt.savefig('task1_terrain.png', dpi=150)
plt.show()

```

Part 2

```

import sympy as sp
from scipy.integrate import tplquad
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

x, y, z = sp.symbols('x y z')

E = x**2*y + 2*y**2*z + 3*z**2*x

total_mass = sp.integrate(
    sp.integrate(
        sp.integrate(E, (x, 0, 2)),
        (y, 0, 2)),
    (z, 0, 2))
print(f"Total mass (symbolic): {total_mass}")

Mx = sp.integrate(sp.integrate(sp.integrate(x*E, (x,0,2)), (y,0,2)), (z,0,2))
My = sp.integrate(sp.integrate(sp.integrate(y*E, (x,0,2)), (y,0,2)), (z,0,2))
Mz = sp.integrate(sp.integrate(sp.integrate(z*E, (x,0,2)), (y,0,2)), (z,0,2))

x_bar = Mx / total_mass
y_bar = My / total_mass
z_bar = Mz / total_mass
print(f"Center of mass (symbolic): ({x_bar}, {y_bar}, {z_bar})")

def density(zv, yv, xv):
    return xv**2*yv + 2*yv**2*zv + 3*zv**2*xv

def density_x(zv, yv, xv):
    return xv * (xv**2*yv + 2*yv**2*zv + 3*zv**2*xv)

```



```

def density_y(zv, yv, xv):
    return yv * (xv**2*yv + 2*yv**2*zv + 3*zv**2*xv)

def density_z(zv, yv, xv):
    return zv * (xv**2*yv + 2*yv**2*zv + 3*zv**2*xv)

M_num, _ = tplquad(density, 0, 2, 0, 2, 0, 2)
Mx_num, _ = tplquad(density_x, 0, 2, 0, 2, 0, 2)
My_num, _ = tplquad(density_y, 0, 2, 0, 2, 0, 2)
Mz_num, _ = tplquad(density_z, 0, 2, 0, 2, 0, 2)

print(f"Total mass (scipy): {M_num:.6f}")
print(f"Center of mass (scipy): ({Mx_num/M_num:.6f}, {My_num/M_num:.6f}, {Mz_num/M_num:.6f})")

fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')

corners = [
    [0,0,0], [2,0,0], [2,2,0], [0,2,0],
    [0,0,2], [2,0,2], [2,2,2], [0,2,2]
]
edges = [(0,1), (1,2), (2,3), (3,0),
         (4,5), (5,6), (6,7), (7,4),
         (0,4), (1,5), (2,6), (3,7)]

for e in edges:
    xs = [corners[e[0]][0], corners[e[1]][0]]
    ys = [corners[e[0]][1], corners[e[1]][1]]
    zs = [corners[e[0]][2], corners[e[1]][2]]
    ax.plot(xs, ys, zs, 'b-', alpha=0.4, linewidth=1.5)

cx, cy, cz = float(x_bar), float(y_bar), float(z_bar)
ax.scatter([cx], [cy], [cz], color='red', s=150, zorder=10,
           label=f'CoM ({cx:.2f}, {cy:.2f}, {cz:.2f})')
ax.text(cx+0.1, cy+0.1, cz+0.1,
        f'CoM\n({cx:.2f},{cy:.2f},{cz:.2f})', fontsize=9, color='red')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D Bounding Volume [0,2]3 with Center of Mass')
ax.legend()
plt.tight_layout()
plt.show()

```

Part 3

```

import sympy as sp
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x, y, z = sp.symbols('x y z')

```

```

E = x**2*y + 2*y**2*z + 3*z**2*x

Fx = sp.diff(E, x)
Fy = sp.diff(E, y)
Fz = sp.diff(E, z)
print(f"F = ({Fx}, {Fy}, {Fz})")

curl_x = sp.diff(Fz, y) - sp.diff(Fy, z)
curl_y = sp.diff(Fx, z) - sp.diff(Fz, x)
curl_z = sp.diff(Fy, x) - sp.diff(Fx, y)

print(f"Curl of F: ({sp.simplify(curl_x)}, {sp.simplify(curl_y)}, {sp.simplify(curl_z)})")
print("Since curl F = (0, 0, 0), F is conservative.")

E_start = E.subs([(x,0), (y,0), (z,0)])
E_end = E.subs([(x,2), (y,2), (z,2)])
work = E_end - E_start
print(f"E(0,0,0) = {E_start}")
print(f"E(2,2,2) = {E_end}")
print(f"Total energy transferred = {work}")

Fx_num = sp.lambdify((x,y,z), Fx, 'numpy')
Fy_num = sp.lambdify((x,y,z), Fy, 'numpy')
Fz_num = sp.lambdify((x,y,z), Fz, 'numpy')

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

t = np.linspace(0, 2, 100)
ax.plot(t, t, t, 'r-', linewidth=3, label='Spaceship path')
ax.scatter([0], [0], [0], color='green', s=100, zorder=10, label='Start (0,0,0)')
ax.scatter([2], [2], [2], color='blue', s=100, zorder=10, label='End (2,2,2)')

gi = np.linspace(0, 2, 5)
GX, GY, GZ = np.meshgrid(gi, gi, gi)
U = Fx_num(GX, GY, GZ)
V = Fy_num(GX, GY, GZ)
W = Fz_num(GX, GY, GZ)

mag = np.sqrt(U**2 + V**2 + W**2) + 1e-10
ax.quiver(GX, GY, GZ,
          U/mag*0.3, V/mag*0.3, W/mag*0.3,
          color='gray', alpha=0.5, length=0.25, linewidth=0.8)

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title("Spaceship Path and Force Field F = ∇E")
ax.legend()
plt.tight_layout()
plt.show()

```

Part 4

```

import sympy as sp
from scipy.integrate import tplquad

```

```

x, y, z = sp.symbols("x y z")
E = x**2*y + 2*y**2*z + 3*z**2*x

```

```

# gradient of E gives F
Fx = sp.diff(E, x)
Fy = sp.diff(E, y)
Fz = sp.diff(E, z)

# divergence of F is the Laplacian of E
div_F = sp.diff(Fx, x) + sp.diff(Fy, y) + sp.diff(Fz, z)
# div_F = 6x + 2y + 4z

# volume integral of divergence over [0,2]^3
flux = sp.integrate(sp.integrate(
    sp.integrate(div_F, (x,0,2)), (y,0,2)), (z,0,2))
# flux = 96

# numerical check with scipy
def div_func(zv, yv, xv):
    return 2*yv + 4*zv + 6*xv # the divergence expression

flux_num, err = tplquad(div_func, 0, 2, 0, 2, 0, 2)
# flux_num = 96.000000 (error estimate: 1.07e-12)

```

6. Conclusion:

- This project demonstrated how multivariable calculus is applied in 3D game engines.
- The scalar field was used to model terrain shading, mass distribution, motion, and volumetric lighting.
- Gradients helped to compute surface normals and lighting effects.
- Triple integrals were used to calculate mass and center of mass.
- The force field derived from the gradient was shown to be conservative which made the energy transfer path independent.
- The divergence theorem was used to calculate volumetric light emission in an efficient way and the results were verified by using Python.
- This project demonstrated how calculus directly supports realistic physics and rendering in game development.

7. Contributions:

- Hiba Asif: Task 1 (terrain and illumination), introduction, flowchart, and report formatting.
- Juhayra Khan: Task 2 and task 3 (physics engine, center of mass, particle systems, line integrals).

- Laiba Mubashir: Task 4 (volumetric lighting), code annexure, conclusions, references, and final proofreading.

8. References:

- Stewart, J. Calculus: Early Transcendentals. Cengage Learning.
- Larson, R., Hostetler, R. P., & Edwards, B. H. Calculus: Early Transcendental Functions. Houghton Mifflin.
- Thomas, G. B., Weir, M. D., & Hass, J. Thomas' Calculus. Pearson.
- FAST NUCES Islamabad Multivariable Calculus Lecture Notes (MT1008)